ADDENDUM — UPDATES BASED ON PHASE 1 & 2 RESULTS

Additions for Phase 3 & 4

To Strengthen the Paper for VMV Submission

Mohamed Hafez

Read This First

Phase 1 baselines are solid and match published MedMNIST benchmarks. Phase 2 reveals that standard TTA is unpredictable rather than uniformly harmful — it helps on most datasets but hurts on PneumoniaMNIST. This addendum specifies the additions needed in Phase 3 and Phase 4 to make the paper competitive at VMV. Do not replace the main proposal — these additions sit on top of it.

Overview — Five Additions

Each addition below has its own section in this document explaining what, why, how, and what the result should look like. Read your assigned section carefully before starting.

Addition	Phase	Owner	Effort	Why It Matters
1. Inference Time Measurement	Phase 3	All students	1 day	Required metric. Reviewers reject TTA papers without latency numbers.
2. Temperature Scaling (TS)	Phase 3	All students	½ day	Standard calibration baseline. Without it, reviewers ask 'why not just use TS?'
3. Confidence Strip × 3 Images	Phase 3	S5	1 day	The visual computing contribution that makes this paper a VMV paper, not an ML paper.
4. Statistical Significance	Phase 4	All students	1 day	Small accuracy gaps need McNemar + Wilcoxon to be defensible.
5. Save Per-Image Predictions	Phase 3	All students	0 days	One extra line per experiment — enables statistical tests later. Do it now.

Important Reframing — Read Carefully

Phase 2 shows standard TTA helps 5 of 6 datasets, not hurts. We are pivoting the paper's headline from 'TTA is not better' to 'TTA is unpredictable, and uncertainty weighting provides a safety net that also improves calibration.' This is a stronger paper. The data supports it. Use this framing in any writing you do from now on.

Addition 1 — Inference Time Measurement

PHASE 3 — ALL STUDENTS

What you are measuring

You are measuring the **wall-clock time, in milliseconds, to produce one final prediction for one original test image** — including all 10 augmented views, all 10 forward passes, and the fusion step. The 10 views are not 10 separate measurements. They are part of the cost of producing one prediction.

Concretely, one inference is:

- Take one test image
- Generate N=10 augmented copies
- Run all 10 through the model (10 forward passes)
- Combine the 10 softmax outputs using your strategy (entropy, max-prob, etc.)
- Output one final predicted class

Time it: start a timer at step 1, stop at step 5. That is one inference. Average across the whole test set to get ms per image.

Why it matters

If entropy weighting gives a 0.5% accuracy gain but is 10× slower than baseline, no clinician deploys it. The cost number is what turns a benchmark into a deployment recommendation. VMV reviewers will look for this — papers without latency numbers get rejected on the question "why would anyone use this?"

How to do it

```
import time
import torch

# CRITICAL: warm-up first (3-5 batches) to discard CUDA kernel compilation time
with torch.no_grad():
    for i, (img, _) in enumerate(test_loader):
        _ = tta_predict(model, img.to(device), N=10, strategy='entropy')
        if i >= 5: break

# Now measure
start = time.perf_counter()
with torch.no_grad():
    for img, label in test_loader:
        pred = tta_predict(model, img.to(device), N=10, strategy='entropy')
elapsed_total = time.perf_counter() - start

ms_per_image = (elapsed_total / len(test_dataset)) * 1000
print(f'{ms_per_image:.2f} ms/image')
```

Rules you must follow

- **GPU consistency:** Same GPU for every measurement — otherwise numbers are not comparable.
- **Warm-up required:** Always warm up the GPU (3–5 batches) before starting the timer.
- **No grad:** Set torch.no_grad() so gradient memory doesn't slow you down.
- **Use perf_counter:** Use time.perf_counter(), not time.time(). It is higher precision.
- **Single run:** Measure once for each (dataset × strategy × N) cell. Single run is enough — these are deterministic.

What the result looks like

Fill the Inf.ms column for every cell in Sheets 2 (Standard TTA) and 3 (Weighted TTA). Example for your row:

Dataset	Baseline (N=1)	N=5	N=10	N=20	N=50
PathMNIST (example)	3.8 ms	19.5 ms	38.2 ms	75.1 ms	187 ms

Sanity check: N=10 inference time should be roughly 10× the N=1 baseline time. If it's much more (e.g. 50×), the augmentation step is the bottleneck — investigate. If it's much less (e.g. 3×), you may not be running the forward passes properly.

Addition 2 — Temperature Scaling (TS)

PHASE 3 — ALL STUDENTS

What temperature scaling is, plainly

Modern neural networks are systematically overconfident. The model might say "I'm 99% sure this is pneumonia" but only be correct 80% of the time on images where it claims that confidence level. This is exactly what ECE measures.

Temperature scaling is a **one-parameter fix** applied **after training**. You divide the model's pre-softmax logits by a single scalar T before applying softmax:

```
# Without temperature scaling
probs = softmax(logits)

# With temperature scaling
probs = softmax(logits / T)     # T > 1 makes the model less confident
```

- If T = 1, nothing changes.
- If T > 1, the logits get squeezed toward zero, softmax flattens, confidence drops.
- If T < 1, confidence rises.

Why TS cannot hurt accuracy

Dividing every logit by the same T shrinks them all proportionally. The biggest logit stays the biggest, so argmax(logits/T) = argmax(logits) for any T > 0. The predicted class never changes — only the confidence number changes. TS is mathematically guaranteed to leave accuracy untouched while improving calibration.

Why we are adding it

Temperature scaling is **the standard calibration baseline**. Every reviewer working on calibration knows it. The first question a reviewer will ask is: "Your method improves ECE — but so does temperature scaling, which is free. Why use uncertainty-weighted TTA instead?"

If we can't answer that, the paper gets rejected. If we can — by including TS as a comparison row — the paper gets accepted. **We must add it.**

Two new columns in Phase 3 (Sheet 3)

You add two new columns to your Weighted TTA results sheet:

Column	What it is	How to compute
TS Only	Single forward pass on each test image, but applying T before softmax. NO augmentation, NO TTA loop.	probs = softmax(model(img) / T) — that's it. One pass per image.
TS + Entropy TTA	Full entropy-weighted TTA pipeline, but every softmax inside it uses logits/T instead of raw logits.	Run your existing entropy TTA function, but replace softmax(logits) with softmax(logits/T) everywhere it appears.

Step 1: Fit T on the validation set (do this once per dataset)

T is learned by minimizing NLL on the validation split. This takes about 2 minutes per dataset on GPU. You do it once and save the T value.

```
import torch
import torch.nn as nn
import torch.optim as optim

class TemperatureScaler(nn.Module):
    def __init__(self):
        super().__init__()
        self.temperature = nn.Parameter(torch.ones(1) * 1.5)
    def forward(self, logits):
        return logits / self.temperature

def fit_temperature(model, val_loader, device):
    model.eval()
    scaler = TemperatureScaler().to(device)
    optimizer = optim.LBFGS([scaler.temperature], lr=0.01, max_iter=50)
    criterion = nn.CrossEntropyLoss()

    # Gather all validation logits and labels once
    all_logits, all_labels = [], []
    with torch.no_grad():
```

```
        for img, label in val_loader:
            img = img.to(device)
            label = label.to(device).squeeze().long()
            all_logits.append(model(img))
            all_labels.append(label)
    all_logits = torch.cat(all_logits)
    all_labels = torch.cat(all_labels)

    def closure():
        optimizer.zero_grad()
        loss = criterion(scaler(all_logits), all_labels)
        loss.backward()
        return loss

    optimizer.step(closure)
    T = scaler.temperature.item()
    print(f'Optimal T = {T:.3f}')
    return T

# Call it once, save the number
T_value = fit_temperature(model, val_loader, device)
# Typical values: T = 1.2 to 2.5
```

Save your T value somewhere visible — write it in the Phase 3 sheet as a comment, e.g. 'T = 1.32 for PathMNIST'. You will need it for both new columns.

Step 2: Compute the TS Only column

This is a single forward pass with the calibrated softmax. No augmentation. No TTA.

```
model.eval()
all_preds, all_probs, all_correct = [], [], []
with torch.no_grad():
    for img, label in test_loader:
        img = img.to(device)
        label = label.to(device).squeeze().long()
        logits = model(img)
        probs = torch.softmax(logits / T_value, dim=1)   # <-- calibrated
        pred = probs.argmax(dim=1)
        all_probs.append(probs.cpu())
        all_preds.append(pred.cpu())
        all_correct.append((pred == label).cpu())

# Then compute Acc, ECE, NLL, AUC normally from probs/preds
```

Step 3: Compute the TS + Entropy TTA column

Reuse your existing entropy TTA function — just feed it calibrated probabilities.

```
def tta_predict_with_ts(model, image, T, N=10):
    """Same as your entropy TTA function, but with TS applied."""
    augmented_views = generate_augmentations(image, N=N)
    probs_list = []
    with torch.no_grad():
```

```
        for view in augmented_views:
            logits = model(view.unsqueeze(0))
            probs = torch.softmax(logits / T, dim=1)    # <-- ONLY change
            probs_list.append(probs.squeeze().cpu().numpy())

    probs_arr = np.array(probs_list)
    entropy = -np.sum(probs_arr * np.log(probs_arr + 1e-8), axis=1)
    weights = np.exp(-entropy)
    weights = weights / weights.sum()
    final_probs = (probs_arr * weights[:, None]).sum(axis=0)
    return final_probs
```

What the result looks like

Your Phase 3 row for one dataset will now have 7 columns:

Baseline TTA	Max-Prob	Entropy	Variance	MC Dropout	TS Only	TS + Entropy
All 5 metrics	All 5 metrics	All 5 metrics	All 5 metrics	All 5 metrics	**All 5 metrics**	**All 5 metrics**

Same 5 metrics in every column: Accuracy, ECE, NLL, AUC (binary only), Inference time. No exceptions.

> The expected ECE pattern across columns is: Baseline → Standard TTA → Entropy TTA → TS Only → TS + Entropy, with ECE decreasing roughly monotonically. The final column should give the best calibration. If it doesn't, that's a finding worth investigating.

Addition 3 — Confidence Strip × 3 Images per Modality

PHASE 3 — S5 OWNS

What it is

A two-row figure for a single test image. Top row shows all N=10 augmented views as thumbnails. Bottom row shows a colored bar under each thumbnail, where the bar's height and color intensity encode the entropy weight w_i assigned to that view. This is the visual contribution that makes the paper a VMV paper rather than an ML benchmark.

Why three images, not one

The original plan was one strip per modality. **Three is better.** A single strip could be cherry-picked. Three images per modality (with the same pattern visible across all three) prove the finding is real and stable — and pre-empts the reviewer asking "did you pick a favorable example?"

If the pattern is consistent across 3 images per modality, that's a finding. If it's not consistent, that's also a finding (and you want to know before a reviewer points it out).

What to deliver

- 4 modalities × 3 images each = 12 confidence strips total
- Modalities: PathMNIST (histology), DermaMNIST (dermoscopy), PneumoniaMNIST (X-ray), BloodMNIST (microscopy)
- For each modality, pick 3 correctly-classified images at random — set seed=0,1,2 so others can reproduce
- Save each as a PDF at DPI=300 in figures/strip/
- Final paper figure: arrange in a 4-row × 3-column grid (4 modalities, 3 images each)

How to read the figure

- **Trusted views:** Dark/tall bars = model was confident on this augmented view → it votes loudly
- **Rejected views:** Pale/short bars = model was uncertain → it gets demoted
- **Per-modality story:** Per-modality patterns are the payoff — e.g. on PneumoniaMNIST, vertical flips should get near-zero weight because an upside-down chest is anatomically impossible

Step-by-step implementation

Step 1 — Pick 3 correctly-classified images per modality

```
import torch, numpy as np
import medmnist

def pick_correct_images(model, dataset, device, n_images=3, seeds=[0, 1, 2]):
    """Pick n_images correctly-classified test images with fixed seeds."""
    model.eval()
    selected = []
    for seed in seeds:
        rng = np.random.RandomState(seed)
        indices = rng.permutation(len(dataset))
        for idx in indices:
            img, label = dataset[idx]
            with torch.no_grad():
                pred = model(img.unsqueeze(0).to(device)).argmax().item()
            true_label = label.item() if hasattr(label, 'item') else int(label)
            if pred == true_label:
                selected.append((idx, img, true_label))
                break
    return selected   # list of 3 (index, image_tensor, label) tuples
```

Step 2 — Generate 10 augmented views & compute entropy weights

```
import torch.nn.functional as F
from utils.augmentations import get_augmentation_pipeline

def compute_weights_for_image(model, image, device, N=10):
    augment_list = get_augmentation_pipeline()[:N]   # 10 (fn, name) pairs
    augmented_views, probs_list, aug_names = [], [], []

    model.eval()
    with torch.no_grad():
        for aug_fn, aug_name in augment_list:
            aug_img = aug_fn(image)
            logits = model(aug_img.unsqueeze(0).to(device))
            probs = F.softmax(logits, dim=1).squeeze().cpu().numpy()
            augmented_views.append(aug_img)
            probs_list.append(probs)
            aug_names.append(aug_name)
```

```
    probs_arr = np.array(probs_list)
    entropy = -np.sum(probs_arr * np.log(probs_arr + 1e-8), axis=1)
    weights = np.exp(-entropy)
    weights = weights / weights.sum()
    return augmented_views, weights, aug_names
```

Step 3 — Plot the strip

```
import matplotlib.pyplot as plt
import matplotlib.cm as cm

def plot_confidence_strip(views, weights, aug_names, title, save_path):
    N = len(views)
    fig, axes = plt.subplots(2, N, figsize=(N * 1.6, 3.5),
                             gridspec_kw={'height_ratios': [3, 1]})
    fig.suptitle(title, fontsize=11, fontweight='bold', y=1.02)

    cmap = cm.Blues
    norm_w = (weights - weights.min()) / (weights.max() - weights.min() + 1e-8)

    for i in range(N):
        ax_img = axes[0, i]
        img_np = views[i].permute(1,2,0).numpy()
        img_np = (img_np - img_np.min()) / (img_np.max() - img_np.min())
        if img_np.shape[2] == 1:
            ax_img.imshow(img_np.squeeze(), cmap='gray')
        else:
            ax_img.imshow(img_np)
        ax_img.set_title(aug_names[i], fontsize=7, rotation=30, ha='right')
        ax_img.axis('off')

        ax_bar = axes[1, i]
        ax_bar.bar(0, weights[i], color=cmap(0.3 + 0.7 * norm_w[i]),
                   width=0.6, edgecolor='navy', linewidth=0.5)
        ax_bar.set_ylim(0, weights.max() * 1.15)
        ax_bar.set_xlim(-0.5, 0.5)
        ax_bar.set_xticks([])
        ax_bar.text(0, weights[i] + 0.002, f'{weights[i]:.2f}',
                    ha='center', va='bottom', fontsize=7)
        if i == 0:
            ax_bar.set_ylabel('weight w_i', fontsize=8)
        else:
            ax_bar.set_yticks([])

    plt.tight_layout()
    plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.close()
```

Step 4 — Loop over the 4 modalities × 3 images and save

```
modalities = {
    'PathMNIST':      'Histology',
    'DermaMNIST':     'Dermoscopy',
    'PneumoniaMNIST': 'Chest X-Ray',
    'BloodMNIST':     'Microscopy',
}

for ds_name, modality in modalities.items():
```

```
    model = load_checkpoint(f'checkpoints/{ds_name.lower()}_resnet18.pth')
    test_ds = get_test_dataset(ds_name)
    images = pick_correct_images(model, test_ds, device, n_images=3)

    for img_idx, (idx, img, label) in enumerate(images):
        views, weights, names = compute_weights_for_image(model, img, device)
        title = f'{ds_name} ({modality}) - Sample {img_idx+1} (idx={idx})'
        save_path = f'figures/strip/{ds_name.lower()}_sample{img_idx+1}.pdf'
        plot_confidence_strip(views, weights, names, title, save_path)

print('Done — 12 strip figures generated.')
```

Sanity check — what to look for

Modality	Expected pattern in the weight bars
PathMNIST (Histology)	Color jitter likely receives low weight (color is diagnostic in tissue staining). Rotation/flip should get high weight (histology is rotation-invariant).
DermaMNIST (Dermoscopy)	Brightness shift & color jitter should get LOW weight (skin lesion color is diagnostic). Rotation should be trusted.
PneumoniaMNIST (X-ray)	Vertical flip and horizontal flip should get near-ZERO weight (an upside-down or mirrored chest is anatomically impossible). Slight rotation should be trusted.
BloodMNIST (Microscopy)	Most augmentations should get high weight — blood cells are rotation-invariant and symmetric. This is the easy case.

If the patterns above hold across all 3 images per modality → the figure is the centerpiece of the paper. If patterns are inconsistent between the 3 images → flag it immediately. That's not a failure, it's information you need before writing the paper.

Addition 4 — Statistical Significance

PHASE 4 — ALL STUDENTS

Why we need this

When BreastMNIST goes from 88.46% (standard TTA) to 89.10% (entropy weighting), that's a +0.64pp gain. On a test set of 156 images, +0.64pp = one extra correct image. One image is not a result — unless we prove statistically that it isn't noise. Reviewers reject papers that claim improvements without significance testing.

We use three complementary tools, each answering a different question.

Tool 1 — McNemar's Test (per-dataset)

Question it answers: Did entropy TTA beat standard TTA on this specific dataset, accounting for randomness?

How it works: It compares the two methods image-by-image on the same test set. Looks at the two off-diagonal counts — how many images entropy got right but standard got wrong, vs how many standard got right but entropy got wrong. Asks: is the imbalance significant?

When to run: Once per dataset, comparing the best weighted strategy vs Baseline TTA. Six tests total.

```
from statsmodels.stats.contingency_tables import mcnemar
import numpy as np

# Load saved per-image predictions (saved during Phase 3)
y_true   = np.load(f'predictions/{dataset}_labels.npy')
pred_std = np.load(f'predictions/{dataset}_standard_preds.npy')
pred_ent = np.load(f'predictions/{dataset}_entropy_preds.npy')

std_right = (pred_std == y_true)
ent_right = (pred_ent == y_true)

b01 = ((std_right) & (~ent_right)).sum()   # std right, ent wrong
b10 = ((~std_right) & (ent_right)).sum()   # std wrong, ent right

result = mcnemar([[0, b01], [b10, 0]], exact=False, correction=True)
print(f'{dataset}: std_only={b01}, ent_only={b10}, p-value = {result.pvalue:.4f}')

# p < 0.05 -> entropy TTA significantly different from standard TTA
```

Tool 2 — Wilcoxon Signed-Rank Test (across datasets)

Question it answers: Does entropy TTA consistently beat standard TTA across all 6 modalities, or is the win driven by one or two lucky datasets?

How it works: Takes the 6 paired accuracies (one per dataset) and asks if the ranking is meaningfully one-sided.

When to run: Once, for the whole paper, on the final results table.

```
from scipy.stats import wilcoxon

# 6 paired accuracies from final results table
entropy_accs  = [0.9433, 0.8483, 0.8654, 0.8910, 0.9833, 0.9479]
standard_accs = [0.9415, 0.8449, 0.8606, 0.8846, 0.9830, 0.9479]

stat, p = wilcoxon(entropy_accs, standard_accs)
print(f'Wilcoxon p-value = {p:.4f}')

# Repeat for ECE to show calibration also consistently improves
entropy_eces  = [...]   # filled from your table
standard_eces = [...]
stat_ece, p_ece = wilcoxon(standard_eces, entropy_eces)  # note order: lower is
better for ECE
print(f'ECE Wilcoxon p-value = {p_ece:.4f}')
```

Tool 3 — Multiple Training Seeds (optional but recommended)

Question it answers: Is our baseline number itself stable, or did we get lucky with one training run?

How it works: Retrain each model 3 times with different random seeds (e.g., 0, 42, 123). Run all your Phase 2 & 3 experiments on all 3 checkpoints. Report mean ± std instead of a single number.

Cost: 3× training time. ResNet-18 at 64×64 trains in <30 min per dataset on a single GPU. Total compute = ~9 GPU-hours, parallelizable across students.

> **If time is tight**
>
> Do 3 seeds on 3 representative datasets (PathMNIST as large-multiclass, PneumoniaMNIST as binary, BreastMNIST as small). Single seed on the others. Flag this transparently in the paper's Experimental Setup section. Reviewers accept this if you're honest about it.

What the result looks like — Sheet 7 (Statistical Tests)

Dataset	Best Strategy	p (McNemar)	Significant?	Interpretation
PathMNIST (example)	Entropy	0.003	✓ YES	Entropy beats standard TTA significantly
BreastMNIST (example)	Max-Prob	0.317	✗ NO	Small dataset; gap may be noise

This table goes in the paper as Table 3 (Statistical Significance). Even rows that don't reach significance are honest, reportable findings — small datasets like BreastMNIST may genuinely not have enough samples to detect small effects.

Addition 5 — Save Per-Image Predictions

PHASE 3 — ALL STUDENTS — DO NOW

Why this matters NOW

Statistical tests (Addition 4) need **per-image** predictions — not just the final accuracy number. If you only save "94.15%" for standard TTA, you cannot run McNemar later. You'll need to **rerun the whole experiment**. Save the arrays now. It costs nothing and saves a full rerun in Phase 4.

What to save

For every (dataset × strategy) experiment in Phase 3, save 3 numpy arrays:

- predictions/{dataset}_{strategy}_preds.npy — predicted class for each test image (shape: N_test,)
- predictions/{dataset}_{strategy}_probs.npy — full softmax output for each test image (shape: N_test, num_classes)
- predictions/{dataset}_labels.npy — ground truth labels (shape: N_test,) — save once per dataset

How to save

```
import numpy as np
import os

os.makedirs('predictions', exist_ok=True)

# Inside your TTA evaluation loop, accumulate predictions & probs:
all_preds, all_probs = [], []
for img, _ in test_loader:
    probs = tta_predict(model, img, N=10, strategy='entropy')
    all_probs.append(probs)
    all_preds.append(np.argmax(probs, axis=1))

all_preds = np.concatenate(all_preds)   # shape (N_test,)
all_probs = np.concatenate(all_probs)   # shape (N_test, num_classes)

# Save once per (dataset, strategy)
np.save(f'predictions/{ds}_entropy_preds.npy', all_preds)
np.save(f'predictions/{ds}_entropy_probs.npy', all_probs)

# Save labels once per dataset
np.save(f'predictions/{ds}_labels.npy', test_labels)
```

Folder you should end Phase 3 with

```
predictions/
├── pathmnist_labels.npy
├── pathmnist_baseline_preds.npy
├── pathmnist_baseline_probs.npy
├── pathmnist_entropy_preds.npy
├── pathmnist_entropy_probs.npy
├── pathmnist_maxprob_preds.npy
├── ... (same pattern for all 6 datasets × 7 strategies)
└── pathmnist_ts_entropy_probs.npy
```

Total expected: 6 datasets × 7 strategies × 2 files + 6 label files = 90 numpy files. They are small (<10MB each). Commit them to a shared cloud folder (NOT the git repo — they're large for git).

Final Checklist Before Phase 4 Begins

Tick each item before moving from Phase 3 to Phase 4.

☐	Item	Owner
☐	BloodMNIST ECE column in Phase 2 sheet is corrected (was AUC mistakenly pasted)	**S5**
☐	All Inf.ms cells in Phase 2 sheet are filled	**All**
☐	Per-image predictions saved as numpy files for every (dataset × strategy) cell	**All**
☐	Temperature T value fitted and recorded for every dataset	**All**
☐	TS Only column filled in Phase 3 sheet	**All**

☐	TS + Entropy column filled in Phase 3 sheet	**All**
☐	All Inf.ms cells in Phase 3 sheet are filled	**All**
☐	12 confidence strip figures generated and reviewed by supervisor	**S5**
☐	3 training seeds completed for at least 3 datasets (or all 6 if time permits)	**All / S5**
☐	Phase 3 sheet is 100% filled — no empty cells remain	**All**
☐	BloodMNIST ECE value sanity check: should be ~0.01, NOT ~0.99	**S5**
☐	Supervisor has reviewed all entries before locking the sheet for analysis	**Supervisor**

Key Messages — Read Before Starting

1. The paper's headline has changed

We no longer claim 'TTA is not better.' We now claim: 'TTA is unpredictable across modalities; uncertainty weighting and temperature scaling together provide a safety net and consistently improve calibration.' Reframe any writing you've done with this in mind.

2. Calibration is the primary story now, not accuracy

ECE and NLL are the metrics that tell the strongest story. Accuracy improvements are small (~0.5-2pp) but calibration improvements are large (Entropy TTA reduced BloodMNIST ECE by 43%). Lead with calibration in any analysis and writing.

3. Variance weighting is a deliberate negative finding

On BreastMNIST, variance weighting collapsed accuracy by 5pp. Do not hide this — report it. "Naive variance weighting is unstable on small datasets" is a useful, honest finding that strengthens the paper's rigor.

4. Inference time is not optional

Every TTA paper without latency numbers gets rejected. Fill the Inf.ms column for every cell. Same GPU, warm-up first, time.perf_counter().

5. Save per-image predictions during Phase 3 — not after

If you only save final accuracy numbers, statistical tests (McNemar) become impossible without rerunning. Save the .npy files as you go.

Questions? Ask in the group chat before guessing. A wrong run is more expensive than a 5-minute clarification.

End of addendum. Read alongside the main proposal. — Mohamed Hafez